


Tool Fundamentals & Design Principles

Building Reliable Tools for LLM Agents





Topics

- Function Calling & Schemas
- LLM Tool Selection
- Parsing & Error Handling
- Core Design Principles.



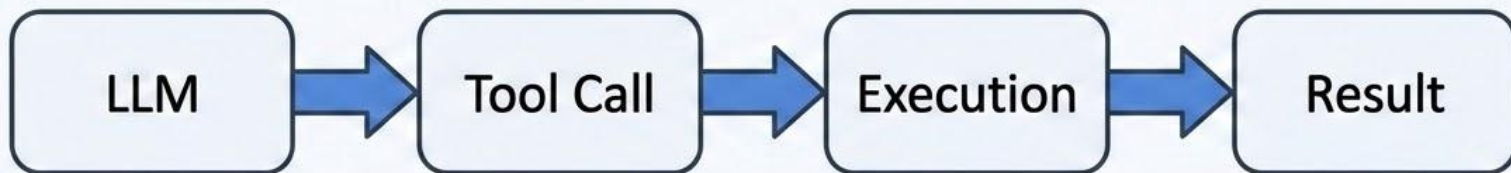
Why Great Tools = Successful Agents



- Enhanced Capability
- Improved Efficiency
- Higher Reliability
- Better User Experience

Bad Tools	vs	Great Tools
● Unreliable, Undocumented ● Complex, Arduous		✓ Stable, Schema-Defined ✓ Intuitive, Scalable, Optimized

Function Calling / Tool Use Basics



- Defines functions as tools for an LLM.
- LLM chooses a tool based on the user request.
- Executes the tool and receives the structured result.

```
def get_current_weather(location, unit="celsius"):
    """Gets the current weather."""
    # Tool execution logic here
    print(f"Fetching weather for {location} in {unit}.")
```

Tool Schemas: JSON Schema vs Pydantic

JSON Schema (Traditional)

```
{
  "name": "search_products",
  "type": "string",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string"
      }
    }
  }
}
```

Pydantic (Python-Native)

```
import pydantic

class SearchProductsParameters(pydantic):
    query: str
    location: str
    strong_type: str
```

- ✓ Strong static typing & runtime validation
- ✓ Auto-generates readable documentation (e.g., OpenAPI)
- ✓ Seamless integration with Python ecosystems (LLM frameworks)

How LLMs Decide Which Tool to Call

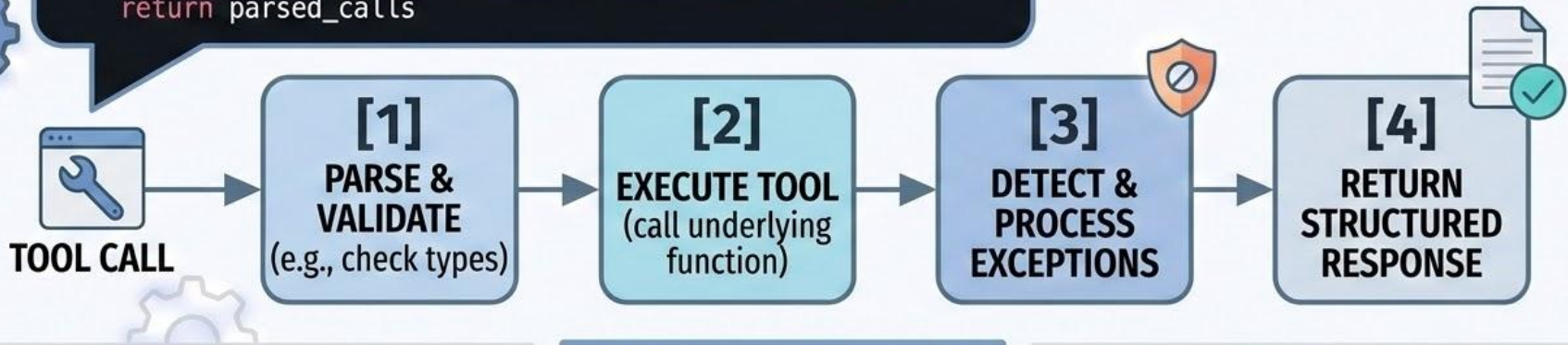


Example Tool Description (Highlighted)

```
def search_products(query: str):  
    """Description: Use this tool to find  
    relevant products..."""
```

Parsing Tool Calls & Graceful Error Handling

```
def handle_tool_calls(tool_calls):  
    """Parses and executes tool calls with error handling."""  
    parsed_calls = []  
    for tool_call in tool_calls:  
        try:  
            call_info = parse_and_validate(tool_call)  
            parsed_calls.append(call_info)  
        except ValidationError as e:  
            handle_parsing_error(tool_call, e)  
        except Exception as e:  
            handle_execution_error(tool_call, e)  
    return parsed_calls
```



Tool Design Principles – Part 1



1. One Tool = One Job

Focus each tool on a single, atomic task.

```
def get_and_validate_user(user_id): BEFORE  
    # Fetches user, checks status, validates  
    permissions...  
    ...
```

```
def get_user_profile(user_id): AFTER  
    """Fetches raw profile data."""  
    ...
```

```
def is_user_active(user_id):  
    """Checks user status."""  
    ...
```



2. Clear LLM-Readable Docstrings

Ensure descriptions explain **what** and *how** to use the tool.

```
def lookup_product(pid): BEFORE  
    # finds the item  
    res = db.fetch(pid)  
    return res
```

```
def lookup_product(product_id: str): AFTER  
    """Use this tool to find a product by its  
    unique product_id string. Returns a  
    detailed dictionary of product  
    information."""  
    res = product_db.fetch(product_id)  
    return res
```

Tool Design Principles – Part 2



[3] RETURN STRUCTURED DATA (Pydantic)

Ensure responses are machine-readable and type-validated.

```
def get_and_validate(user_id):  
    # Instable user account...  
    return [[list, "", 2,3]]  
    ...
```

BEFORE

```
def tool_response1(UserResponseModel):  
    return = UserResponseModel()  
    return "Fetches and returns validated user  
    data as a Pydantic model."
```

AFTER

BENEFITS:

- *Guarantees correct data structures for LLMs.
- *Simplifies downstream logic.
- *Self-documenting.*



[4] BUILD RESILIENCE INSIDE TOOLS

Tools should handle internal complexity and only return stable results.

```
def tool_make():  
    api = makes. api.call("&API"))
```

BEFORE

```
def tool_call(strr):  
    try: api == 01 as e:  
        # Interacts with external API, e)  
        call = connecton_isses  
    except Exception as e:  
        handle_execution_error(tool_call, e)  
    return parse_call
```

AFTER

BENEFITS:

- *Prevents tool failures from stopping the conversation.
- *Simplifies user instructions.
- *Consistent, predictable output.*

Common Anti-Patterns & Pitfalls



[1] MONOLITHIC TOOLS

Expl: Combining multiple unrelated actions.

WHY IT'S BAD: Unpredictable behavior, hard to maintain.

[2] IMPLICIT ASSUMPTIONS

Expl: Relying on specific execution contexts.

WHY IT'S BAD: Tool fails in new scenarios.

[3] HARDCODED SECRETS

Expl: Placing API keys/credentials directly in code.

WHY IT'S BAD: Major security risk, difficult to update.



[4] NON-STANDARD RESPONSES

Expl: Returning inconsistent or unstructured data formats.

WHY IT'S BAD: Downstream logic fails to parse output.

[5] MISSING ERROR HANDLING

Expl: Allowing tools to crash with standard exceptions.

WHY IT'S BAD: Breaks entire workflow, no graceful recovery.

[6] OPAQUE INPUT SCHEMAS

Expl: Using loose or undefined validation for inputs.

WHY IT'S BAD: Easy for LLMs to generate invalid data.

Real-World Tool Examples



[1] SEARCH TOOL

Finds relevant documents by querying a semantic search API.



[2] DATABASE QUERY

Executes optimized SQL commands to retrieve specific data points.



[3] FILE PROCESSOR

Analyzes and transforms local files for structured consumption.



[4] QUEUE EXPLORER

Monitors and interacts with asynchronous message queues (Kafka, Celery).

Summary

- Tool design fundamentals: Focus, Predictability, Composition.
- Anti-patterns to avoid: Monolithic, Hardcoded secrets, Opaque schemas.
- Implement error handling and rigid I/O schemas.
- Examples: DB Queries, Queue Explorers, File Processors.

Thank You

